

Rapport Système de Gestion de Tickets (Partie 2)

Présenté par : Dramé, Abdramane (DRAA04070300)

Section I - Critique du diagramme de classe initial

En analysant le diagramme de la partie 1, j'ai remarqué plusieurs problèmes qui ne respectent pas l'énoncé ni les bonnes pratiques de conception. Il y a une incohérence entre l'énoncé et le diagramme. Le texte parle de développeurs qui assignent et qui ferment les tickets, mais le diagramme utilise une classe Admin qui n'est jamais mentionnée dans l'énoncé. Ce sont les développeurs qui ont accès au ticket et qui peuvent l'assigner à un autre membre de l'équipe. Le diagramme ne supporte pas les pièces jointes. L'énoncé demande clairement que l'utilisateur puisse fournir une description détaillée incluant du texte, des captures d'écran ou des vidéos. Or, dans le diagramme initial, la description est juste un String, ce qui ne peut pas contenir une image ou une vidéo. La méthode addComment ne stocke rien. Le diagramme prévoit une méthode addComment mais aucun attribut n'est déclaré dans la classe Ticket pour stocker les commentaires. Le commentaire est donc perdu. Le statut et la priorité sont des String. Cela veut dire qu'on peut écrire "OUVERT", "ouvert" ou "Ouvvert" et le programme va accepter n'importe quoi. Il n'y a aucune garantie que le statut soit valide. La liste des tickets est globale. Dans la partie 1, j'ai dû créer une liste tousLesTickets en dehors des classes car le diagramme ne précise pas où sont stockés les tickets. Cela crée un couplage fort entre les classes. La méthode viewAllTickets prend la liste en paramètre. C'est bizarre parce que l'Admin doit recevoir la liste, mais il ne sait pas d'où elle vient. Il n'y a pas de classe qui centralise les tickets.

Section II - Améliorations apportées au diagramme et justifications

Voici le tableau des changements apportés au diagramme avec leurs justifications.

Changement	Justification
Admin renommé en Developpeur	L'énoncé parle de développeurs, pas d'Admin. En appliquant le principe Expert en information, c'est le développeur qui détient la connaissance pour assigner, valider et fermer un ticket. La classe Admin n'avait donc aucune raison d'exister.
Developpeur hérite de User	Un développeur EST un utilisateur spécialisé, ce qui applique le principe de Polymorphisme : il réutilise les méthodes de User et ajoute les siennes. Un Developpeur peut ainsi être traité comme un User si nécessaire.
Classe abstraite DescriptionTicket ajoutée	En appliquant le principe Protégé des variations, on isole ce qui change (le chargement d'un texte, d'une image ou d'une vidéo) du reste du code. La classe abstraite définit le contrat commun à tous les types de descriptions.
Sous-classes TextObject, ImageObject, VideoObject	Chaque sous-classe implémente load() selon son type, ce qui applique le principe de Polymorphisme. On peut appeler description.load() sans savoir de quel type il s'agit, et ajouter un nouveau type sans modifier le reste du code.
DescriptionFactory ajoutée	La logique de création ne trouve pas naturellement sa place dans Ticket ni dans DescriptionTicket. En appliquant la Fabrication pure, on crée une classe artificielle dédiée uniquement à la création des descriptions, ce qui rend le code plus clair et plus maintenable.
Statut remplacé par Enum Statut	En appliquant le principe Protégé des variations, les valeurs possibles sont fermées et protégées. On ne peut plus écrire "ouvert" ou "Ouvert" par erreur, et seules les valeurs OUVERT, ASSIGNÉ, VALIDATION et TERMINÉ sont acceptées.

Priorite remplacé par Enum Priorite	Idem pour la priorité : en appliquant le principe Protégé des variations, seules les valeurs BASSE, NORMALE et HAUTE sont autorisées, ce qui évite les erreurs de saisie.
typeDescription remplacé par Enum TypeDescription	La factory reçoit désormais un type sûr. Cela applique le principe Protégé des variations, car il est impossible de passer un String invalide à la factory.
Commentaires retirés, unifiés avec TextObject	L'énoncé n'en parle pas et TextObject permet déjà d'ajouter du texte libre. En appliquant le principe Forte cohésion, on évite d'avoir deux mécanismes redondants : un seul concept suffit pour gérer le texte.
Attribut createdBy ajouté dans Ticket	En appliquant le principe Expert en information, c'est la classe qui contient l'information qui doit la gérer. Le ticket sait qui l'a créé, donc c'est à lui de stocker cette information.
Attribut assignedTo ajouté dans Ticket	Idem : en appliquant le principe Expert en information, le ticket doit savoir à qui il est assigné, car personne d'autre ne détient cette information de manière aussi directe.
Classe GestionnaireTickets ajoutée	La liste des tickets est centralisée dans une classe dédiée. En appliquant la Fabrication pure et l'Indirection, le gestionnaire sert d'intermédiaire entre le Developpeur et la liste, et évite que le Developpeur manipule directement la liste.
viewAllTickets(gestionnaire) avec paramètre	En appliquant le principe Faible couplage, le Developpeur utilise le gestionnaire pour récupérer les tickets au lieu de dépendre directement de la liste. Cela limite les dépendances entre les classes.
Attribut role retiré de User	Avec l'héritage Developpeur User, le rôle est désormais implicite par la classe. En appliquant le principe Forte cohésion, on évite la redondance et

	on garantit qu'une classe n'a que les attributs nécessaires à sa responsabilité.
--	--

Section IV - Leçons apprises

Au terme de ce travail, ce que l'on retient, c'est que le diagramme initial avait plusieurs problèmes qui ne respectaient pas l'énoncé ni les bonnes pratiques de conception. En appliquant les principes GRASP, on a pu améliorer le diagramme pour supporter les pièces jointes, protéger les statuts avec des énumérations, et organiser le code de manière plus propre. Les principes les plus importants dans ce travail sont l'Expert en information, la Fabrication pure, le Faible couplage et le Protégé des variations. En effet, l'Expert en information nous a permis de placer chaque responsabilité dans la bonne classe, la Fabrication pure nous a permis de créer des classes utilitaires (Factory et Gestionnaire) sans surcharger les classes métier, le Faible couplage nous a évité de dépendre directement de la liste des tickets, et le Protégé des variations nous a permis d'utiliser des énumérations et du polymorphisme pour rendre le code plus robuste. J'ai aussi appris qu'il faut toujours analyser l'énoncé avant de concevoir un diagramme. En effet, plusieurs problèmes venaient du fait que le diagramme initial ne respectait pas le texte. Une bonne conception commence toujours par une bonne compréhension du besoin. De plus, j'ai compris l'importance du polymorphisme et des design patterns comme la Factory. Ces outils permettent d'écrire du code plus propre et plus facile à faire évoluer. Par exemple, si on devait ajouter un type audio dans le futur, il suffirait de créer une sous-classe AudioObject et de l'ajouter dans la factory, sans toucher au reste du code.

Enfin, ce travail m'a montré que la conception et le code doivent aller ensemble. Un bon diagramme donne un bon code, et un bon code respecte le diagramme. C'est cette cohérence qui fait la qualité d'un projet.